TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TẦNG 2 Chọn đội tuyển Tinh dự thi Quốc gia

SỐ HỌC
THUẬT TOÁN

Sàng tuyến tính · Hàm số học · Nghịch đảo modulo · Miller – Rabin
Tổ hợp mod p · Legendre · Lũy thừa ma trận · Digit DP

SÀNG 1..120 · gạch chéo theo ước nguyên tố nhỏ nhất

1	2	3	~~4~~	5	~~6~~	7	~~8~~	~~9~~	~~10~~	11	~~12~~
13	~~14~~	~~15~~	~~16~~	17	~~18~~	19	~~20~~	~~21~~	~~22~~	23	~~24~~
~~25~~	~~26~~	~~27~~	~~28~~	29	~~30~~	31	~~32~~	~~33~~	~~34~~	~~35~~	~~36~~
37	~~38~~	~~39~~	~~40~~	41	~~42~~	43	~~44~~	~~45~~	~~46~~	47	~~48~~
~~49~~	~~50~~	~~51~~	~~52~~	53	~~54~~	~~55~~	~~56~~	~~57~~	~~58~~	59	~~60~~
61	~~62~~	~~63~~	~~64~~	~~65~~	~~66~~	67	~~68~~	~~69~~	~~70~~	71	~~72~~
73	~~74~~	~~75~~	~~76~~	~~77~~	~~78~~	79	~~80~~	~~81~~	~~82~~	83	~~84~~
~~85~~	~~86~~	~~87~~	~~88~~	89	~~90~~	~~91~~	~~92~~	~~93~~	~~94~~	~~95~~	~~96~~
97	~~98~~	~~99~~	~~100~~	101	~~102~~	103	~~104~~	~~105~~	~~106~~	107	~~108~~
109	~~110~~	~~111~~	~~112~~	113	~~114~~	~~115~~	~~116~~	~~117~~	~~118~~	~~119~~	~~120~~

$a^{p-1} \equiv 1 \pmod p$

$\varphi(n) = n \prod (1 - 1/p)$
p chạy qua ước nguyên tố

$v_p(n!) = \Sigma \lfloor n / p^k \rfloor$
công thức Legendre

$C(n, k) = n! / (k!\,(n-k)!)$
trong O(1) theo mod p

$ax + by = gcd(a, b)$
Euclid mở rộng

$F(10^{18}) \bmod p \leftarrow [1\ 1;\ 1\ 0]^n$ 60 phép bình phương

01	02	03	04	05	06
Sàng nâng cao	Hàm số học	Modulo nâng cao	Tổ hợp mod p	Giai thừa và số mũ	Ma trận · Digit DP

TẬP 1 · LÝ THUYẾT — 20 bài học · 6 chương · chứng minh, chạy tay, số đo thật

Nguyên tố · Ước số · Đồng dư · Tổ hợp · Chữ số

Tổng hợp và biên soạn: Lê Thành Nam

Lê Thành Nam

LỜI NÓI ĐẦU

Số học là "mỏ vàng" của các đề thi chọn đội tuyển học sinh giỏi Tin học. Hầu như đề nào cũng có ít nhất một bài cần tới số nguyên tố, ước số, phép chia lấy dư hay tổ hợp. Điều đặc biệt của chủ đề này là **ràng buộc rất lớn**: n tới 10^7 cho các bài sàng, tới 10^{12} cho sàng đoạn, tới 10^{18} cho lũy thừa, Fibonacci hay đếm chữ số. Với những con số như vậy, mọi vòng lặp "thử hết" đều vô vọng; chỉ những công thức và thuật toán đúng mới cho kết quả trong một giây.

Cuốn sách này là **Tầng 2** của bộ tài liệu Số học Thuật toán, dành cho học sinh ôn thi **chọn đội tuyển Tỉnh dự thi học sinh giỏi Quốc gia**. Sách giả định em đã biết kiểm tra nguyên tố bằng cách chia thử tới √n, sàng Eratosthenes cơ bản, ƯCLN bằng thuật toán Euclid và phép lấy dư cơ bản. Nội dung được sắp xếp theo đúng trình tự từ dễ tới khó:

- **Sàng nâng cao** (Chương 1): sàng tuyến tính O(n), ước nguyên tố nhỏ nhất để phân tích thừa số trong O(log n), sàng đoạn [L, R] với R tới 10^{12}, sàng số ước và tổng ước qua tổng điều hòa.
- **Hàm số học** (Chương 2): số ước τ(n), tổng ước σ(n), hàm phi Euler và sàng phi, hàm nhân tính, hàm Möbius.
- **Modulo nâng cao** (Chương 3): lũy thừa nhanh và định lí Fermat nhỏ, Euclid mở rộng và nghịch đảo modulo, phương trình Diophantine ax + by = c, nhân modulo tránh tràn số và kiểm tra nguyên tố Miller – Rabin, định lí Euler và lũy thừa tầng a^(b^c).
- **Tổ hợp modulo p** (Chương 4): giai thừa và nghịch đảo giai thừa, C(n, k) trong O(1), nghịch đảo của 1..n trong O(n), chia kẹo Euclid, số Catalan, bao hàm – loại trừ.
- **Giai thừa và số mũ** (Chương 5): công thức Legendre, số chữ số 0 tận cùng, chữ số khác 0 cuối cùng của n!.
- **Hai kĩ thuật khác** (Chương 6): lũy thừa ma trận (Fibonacci thứ 10^{18}) và quy hoạch động chữ số (Digit DP).

Bộ tài liệu gồm ba tập đi cùng nhau:

- **Tập 1: Lý thuyết** (cuốn sách này) gồm 20 bài học chia thành 6 chương. Mỗi bài học trình bày đầy đủ lý thuyết trước (định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình vẽ), rồi mới vào một bài toán lập trình với cách chậm, cách nhanh và số đo thời gian thật.
- **Tập 2: Bài tập** gồm 60 bài: mỗi bài học có ba bài tập ở ba mức độ, đề viết theo đúng thể thức đề thi học sinh giỏi, ràng buộc chia theo subtask.
- **Tập 3: Hướng dẫn giải** phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm thực tế của từng cách làm, kể cả những cách sai hoặc đúng nhưng chậm.

Mọi con số trong sách (thời gian chạy, kết quả chạy tay, đáp số các câu hỏi tự kiểm tra) đều là số đo và kết quả thật. Mọi chương trình đều đã được đối chiếu với lời giải vét cạn (nhiều khi viết bằng Python với số nguyên lớn) trên hàng trăm bộ dữ liệu ngẫu nhiên.

Lê Thành Nam

Hai điều cần nhớ khi học chủ đề này. Thứ nhất, **chứng minh quan trọng không kém chương trình**: một công thức số học nhớ sai một chỉ số ($p - 1$ hay p, $\lfloor n/p \rfloor$ hay $\lceil n/p \rceil$) sẽ cho đáp số sai mà không test nhỏ nào phát hiện được, chỉ hiểu vì sao công thức đúng mới tránh được. Thứ hai, **tràn số là kẻ thù số một**: tích hai số cỡ 10^9 đã gần giới hạn của long long, tích hai số cỡ 10^{18} thì vượt xa. Sách luôn chỉ rõ chỗ nào có thể tràn và cách xử lí.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học gồm hai phần lớn:

Phần	Nội dung
Mục tiêu, Khởi động	Những gì em sẽ làm được sau bài học, và một tình huống dẫn tới ý tưởng chính, thường kèm số đo cho thấy cách làm tự nhiên chậm tới mức nào.
PHẦN LÝ THUYẾT	Định nghĩa, định lí, chứng minh, ví dụ chạy tay, hình minh họa. Đọc phần này với giấy bút, tự tính lại các ví dụ.
PHẦN BÀI TOÁN	Một bài toán lập trình dùng đúng lý thuyết vừa học: cách chậm (kèm số đo thời gian thật), cách nhanh, chương trình hoàn chỉnh, phân tích độ phức tạp.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh: tràn số, sai chỉ số, quên trường hợp đặc biệt.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA	Khái niệm mới, cần hiểu chính xác.
ĐỊNH LÍ, TÍNH CHẤT	Kết quả quan trọng, thường kèm chứng minh.
CHỨNG MINH	Lập luận vì sao định lí đúng.
VÍ DỤ, CHẠY TAY	Chạy tay trên số liệu cụ thể.
CÔNG THỨC	Công thức cần nhớ.
Ý TƯỞNG, THUẬT TOÁN	Hướng giải hoặc các bước của một phương pháp.
BÀI TOÁN	Đề bài lập trình của bài học.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp và số đo thời gian thật.
VÌ SAO ĐÚNG?	Giải thích thêm một bước khó.
LƯU Ý	Chi tiết dễ bỏ sót.
LÓE SÁNG	Ý tưởng then chốt, "chìa khóa" của bài.
MỞ RỘNG	Kiến thức nâng cao, có thể bỏ qua khi đọc lần đầu.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.

Kí hiệu.

Kí hiệu	Ý nghĩa
$\lfloor x \rfloor$, $\lceil x \rceil$	phần nguyên dưới, phần nguyên trên của x
a mod m	số dư của a khi chia cho m, luôn trong $[0, m-1]$
$a \equiv b \pmod m$	a và b có cùng số dư khi chia cho m

Lê Thành Nam

Kí hiệu	Ý nghĩa
gcd(a, b), lcm(a, b)	ước chung lớn nhất, bội chung nhỏ nhất
$v_p(n)$	số mũ của số nguyên tố p trong n
lp[n]	ước nguyên tố nhỏ nhất của n (least prime)
$\tau(n)$, $\sigma(n)$	số ước và tổng các ước dương của n
$\varphi(n)$, $\mu(n)$	hàm phi Euler, hàm Möbius
C(n, k)	số tổ hợp chập k của n
a^{-1}	nghịch đảo của a theo modulo đang xét
log n	$\log_2 n$, trừ khi ghi rõ cơ số

Trong các chương trình, MOD luôn là $10^9 + 7$ trừ khi đề nói khác; kiểu ll là long long. Trong hình vẽ, ô cam thường là đối tượng đang xét, ô xanh lá là kết quả, ô vàng là phần cần chú ý, ô xám là phần bị bỏ qua.

Thời gian chạy. Các số đo trong sách là thời gian CPU trên một máy tính cá nhân thông thường, biên dịch với g++ -O2 -std=c++14. Máy chấm của em có thể nhanh hơn hoặc chậm hơn vài lần, nhưng **tỉ lệ** giữa các cách làm thì không đổi: một thuật toán $O(n\sqrt{n})$ luôn chậm hơn thuật toán O(n log log n) hàng trăm lần khi $n = 10^7$, ở bất kì máy nào.

Lời khuyên khi học. Với mỗi công thức, hãy tự kiểm tra trên vài số nhỏ (n = 12, 30, 36, 100) trước khi đọc chứng minh. Với mỗi chương trình, hãy tự hỏi: giá trị lớn nhất của từng biến là bao nhiêu, phép nhân nào có thể tràn? Và luôn viết một lời giải vét cạn để đối chiếu: trong số học, vét cạn trên $n \le 1000$ bắt được gần hết các lỗi công thức.

Lê Thành Nam

MỤC LỤC

LỜI NÓI ĐẦU **2**

CÁCH HỌC VỚI TÀI LIỆU NÀY **4**

CHƯƠNG 1. SÀNG NÂNG CAO **7**

Bài 1. Sàng Eratosthenes và sàng tuyến tính 7
Bài 2. Ước nguyên tố nhỏ nhất và phân tích nhiều số 16
Bài 3. Sàng đoạn [L, R] 23
Bài 4. Sàng số ước, tổng ước và tổng điều hòa 29

CHƯƠNG 2. HÀM SỐ HỌC **35**

Bài 5. Số ước τ(n) và tổng ước σ(n) 35
Bài 6. Hàm phi Euler 42
Bài 7. Hàm nhân tính và sàng tuyến tính tổng quát 49

CHƯƠNG 3. MODULO NÂNG CAO **55**

Bài 8. Lũy thừa nhanh và định lí Fermat nhỏ 55
Bài 9. Thuật toán Euclid mở rộng và nghịch đảo modulo 62
Bài 10. Phương trình Diophantine tuyến tính 69
Bài 11. Nhân modulo tránh tràn số và kiểm tra nguyên tố Miller – Rabin 75
Bài 12. Định lí Euler và lũy thừa tổng a^(b^c) 81

CHƯƠNG 4. TỔ HỢP MODULO P **87**

Bài 13. Giai thừa, nghịch đảo giai thừa và tổ hợp C(n, k) 87
Bài 14. Chia kẹo Euclid và số Catalan 93
Bài 15. Nguyên lí bao hàm – loại trừ 99

CHƯƠNG 5. GIAI THỪA VÀ SỐ MŨ **105**

Bài 16. Công thức Legendre: số mũ của p trong n! 105
Bài 17. Chữ số 0 tận cùng và chữ số khác 0 cuối cùng của n! 111

CHƯƠNG 6. HAI KĨ THUẬT KHÁC **117**

Bài 18. Lũy thừa ma trận: Fibonacci thứ 10^{18} 117
Bài 19. Quy hoạch động chữ số (Digit DP) 125
Bài 20. Digit DP với tính chia hết 131

PHỤ LỤC: BẢNG TRA CỨU NHANH **138**

Lê Thanh Nam

CHƯƠNG 1

SÀNG NÂNG CAO

*Sàng là công cụ đầu tiên và quan trọng nhất của số học thuật toán: thay vì xét từng số một, ta xử lí **cả đoạn 1..n cùng lúc**. Chương này đi từ sàng Eratosthenes quen thuộc tới sàng tuyến tính (Bài 1), mảng ước nguyên tố nhỏ nhất để phân tích hàng triệu số (Bài 2), sàng trên một đoạn [L, R] nằm rất xa, tới 10^{12} (Bài 3), và kĩ thuật "duyệt bội" dựa trên tổng điều hòa để tính số ước, tổng ước của mọi số (Bài 4).*

■ TRONG CHƯƠNG NÀY

- **Bài 1.** Sàng Eratosthenes và sàng tuyến tính
- **Bài 2.** Ước nguyên tố nhỏ nhất và phân tích nhiều số
- **Bài 3.** Sàng đoạn [L, R]
- **Bài 4.** Sàng số ước, tổng ước và tổng điều hòa

Lê Thanh Nam

BÀI 1 Sàng Eratosthenes và sàng tuyến tính

Học xong bài này, em sẽ:

✓ Chứng minh vì sao kiểm tra nguyên tố chỉ cần chia thử tới √n, và vì sao cách đó quá chậm khi phải xét mọi số tới 10^7.

✓ Cài đặt **sàng Eratosthenes** đúng cách (bắt đầu gạch từ p^2, dùng `vector<bool>`), hiểu vì sao độ phức tạp là O(n log log n).

✓ Hiểu **sàng tuyến tính**: mỗi hợp số bị gạch đúng một lần bởi ước nguyên tố nhỏ nhất của nó; chứng minh độ phức tạp O(n).

✓ Biết so sánh hai cách sàng trên thực tế: thời gian, bộ nhớ, và thông tin thu được (mảng lp).

■ KHỞI ĐỘNG

Đếm các số nguyên tố không vượt quá $n = 10^7$. Cách quen thuộc là kiểm tra từng số bằng chia thử: chương trình chạy mất **3,28 giây**. Sàng Eratosthenes, một thuật toán hơn hai nghìn năm tuổi, làm cùng việc đó trong **0,065 giây**, nhanh hơn khoảng 50 lần. Bí quyết: thay vì hỏi "số này có chia hết cho ai không?", ta đi theo chiều ngược lại: "số nguyên tố này là ước của những số nào?".

PHẦN LÝ THUYẾT

1. Kiểm tra nguyên tố bằng chia thử

■ SỐ NGUYÊN TỐ, HỢP SỐ

Số tự nhiên $p > 1$ là **số nguyên tố** nếu p chỉ có hai ước dương là 1 và p. Số tự nhiên $n > 1$ không nguyên tố là **hợp số**. Số 0 và số 1 không phải số nguyên tố, cũng không phải hợp số.

■ ƯỚC NHỎ NHẤT KHÔNG VƯỢT QUÁ √n

Nếu $n > 1$ là hợp số thì n có một ước d với $2 \le d \le \sqrt{n}$. Hơn nữa, **ước lớn hơn 1 nhỏ nhất của mọi số $n > 1$ là một số nguyên tố**.

■ CHỨNG MINH

Vì n là hợp số nên $n = a \cdot b$ với $2 \le a \le b$. Nếu $a > \sqrt{n}$ thì $a \cdot b \ge a^2 > n$, vô lí; vậy $a \le \sqrt{n}$. Gọi d là ước lớn hơn 1 nhỏ nhất của n. Nếu d là hợp số thì d có ước e với $1 < e < d$, khi đó e cũng là ước của n, mâu thuẫn với d nhỏ nhất. Vậy d nguyên tố.

Lê Thành Nam

Từ định lí suy ra cách kiểm tra quen thuộc: thử chia n cho $d = 2, 3, \ldots, \lfloor\sqrt{n}\rfloor$; nếu không có phép chia hết nào thì n nguyên tố. Một lần kiểm tra tốn $O(\sqrt{n})$. Nhưng nếu phải kiểm tra **mọi** số từ 2 tới n, tổng chi phí là $O(n \cdot \sqrt{n})$ với các số nguyên tố (mỗi số nguyên tố phải thử đủ $\sqrt{n}$ ước), tức khoảng $664\,579 \cdot 3000 \approx 2 \cdot 10^9$ phép chia với $n = 10^7$. Phép chia lại là phép tính chậm nhất của bộ xử lí.

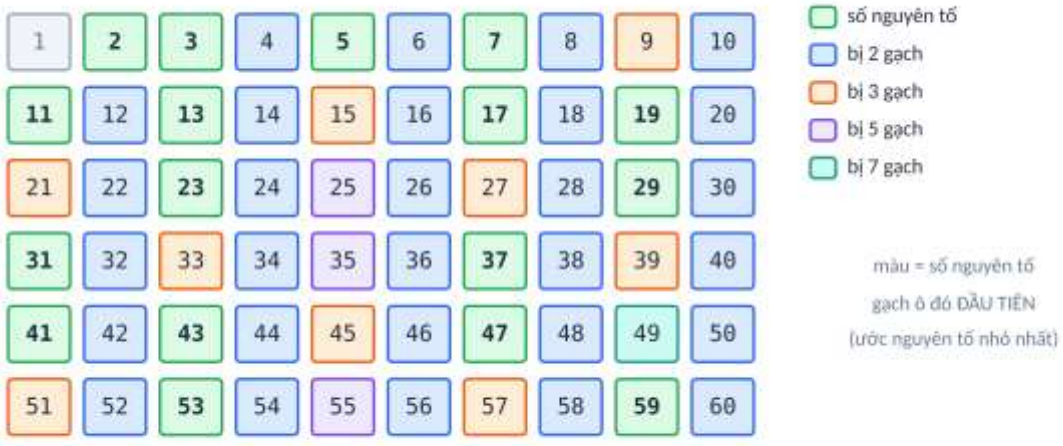
■ ĐIỀU KIỆN DỪNG d·d ≤ n

Viết `d * d <= n` thay cho `d <= sqrt(n)`: tránh sai số của số thực (sqrt(49) có thể trả về 6,9999999). Khi n gần 2^{31}, hãy ép $d \cdot d$ sang `long long` để không tràn.

2. Sàng Eratosthenes

Ý tưởng của Eratosthenes: viết các số từ 2 tới n. Số đầu tiên chưa bị gạch là 2, nó nguyên tố; gạch mọi bội của 2 lớn hơn 2. Số tiếp theo chưa bị gạch là 3, nó nguyên tố; gạch mọi bội của 3. Cứ thế

tiếp tục. Một số còn lại không bị gạch khi và chỉ khi không có số nguyên tố nào nhỏ hơn nó là ước của nó, tức là nó nguyên tố.



Hình 1.1. Sàng tới 60: mỗi hợp số được tô theo số nguyên tố gạch nó đầu tiên (chính là ước nguyên tố nhỏ nhất)

Hai cải tiến quan trọng làm sàng nhanh hơn hẳn:

- **Bắt đầu gạch từ p^2.** Mọi bội $k \cdot p$ với $k < p$ đã bị gạch bởi một ước nguyên tố của k, nhỏ hơn p. Ví dụ khi tới p = 7, các số 14, 21, 28, 35, 42 đã bị gạch bởi 2, 3, 2, 5, 2 rồi; số đầu tiên cần gạch là 49.
- **Chỉ cần $p \le \sqrt{n}$.** Nếu $p^2 > n$ thì không còn bội nào để gạch. Mọi số chưa bị gạch sau khi xét xong các $p \le \sqrt{n}$ đều nguyên tố (định lí ở mục 1).

■ SÀNG ERATOSTHENES

(1) Đánh dấu mọi số từ 2 tới n là "chưa bị gạch".

(2) Với p = 2, 3, ... chừng nào $p \cdot p \le n$: nếu p chưa bị gạch thì gạch p^2, $p^2 + p$, $p^2 + 2p$, ... (không vượt quá n).

(3) Các số chưa bị gạch là số nguyên tố.

▣ Sàng Eratosthenes với vector<bool>

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    cin >> n;
    vector<bool> hop(n + 1, false);        // hop[x] = true: x là hợp số
    for (long long p = 2; p * p <= n; p++)
        if (!hop[p])
            for (long long x = p * p; x <= n; x += p) hop[x] = true;
    long long dem = 0, tong = 0;
    for (int x = 2; x <= n; x++)
        if (!hop[x]) { dem++; tong += x; }
```

Lê Thành Nam

Sàng Eratosthenes với vector<bool>

```
16      cout << dem << ' ' << tong << '\n';
17      return 0;
18  }
```

■ ĐỘ PHỨC TẠP CỦA SÀNG ERATOSTHENES

Tổng số lần gạch không quá $\Sigma_{p \le \sqrt{n}} n/p = O(n \log \log n)$.

■ CHỨNG MINH

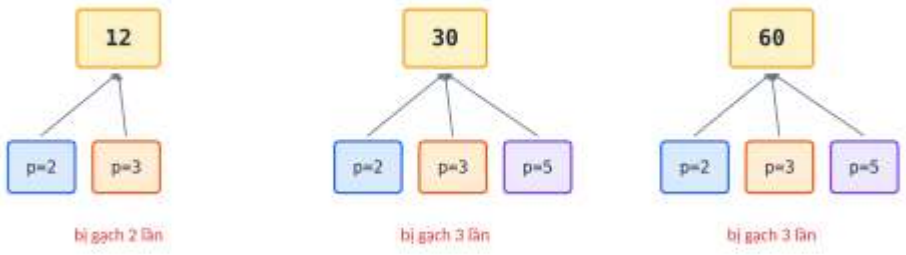
Với mỗi số nguyên tố $p \le \sqrt{n}$, số lần gạch không quá n/p. Tổng các nghịch đảo của số nguyên tố, $1/2 + 1/3 + 1/5 + 1/7 + ... + 1/p$ với $p \le x$, tăng rất chậm: xấp xỉ ln ln x (định lí Mertens, có thể tin dùng mà không cần chứng minh trong sách này). Vậy tổng số lần gạch xấp xỉ $n \cdot \ln \ln \sqrt{n}$. Với $n = 10^7$, $\ln \ln \sqrt{n} \approx 2{,}1$: sàng chỉ tốn khoảng $2n$ bước. Đếm thực tế: sàng tới 10^7 gạch **22.850.051** lần.

■ vector<bool> VÀ BỘ NHỚ

`vector<bool>` lưu mỗi phần tử bằng **một bit**: sàng tới 10^8 chỉ tốn 12,5 MB. Mảng `bool` thường tốn một byte mỗi phần tử (100 MB), mảng `int` tốn 4 byte (400 MB). Ngoài ra, mảng bit nhỏ nằm gọn trong bộ nhớ đệm của bộ xử lí nên duyệt nhanh hơn.

3. Vì sao sàng Eratosthenes chạy ra tuyến tính?

Sàng Eratosthenes làm việc thừa: một hợp số bị gạch **bởi mỗi ước nguyên tố không vượt quá $\sqrt{n}$ của nó**. Số 30 bị gạch ba lần (bởi 2, bởi 3 và bởi 5).



Hình 1.2. Một hợp số bị gạch nhiều lần: mỗi ước nguyên tố gạch nó một lần

Muốn mỗi hợp số chỉ bị gạch **đúng một lần**, ta phải chọn trước "người gạch" duy nhất cho nó. Ứng viên tự nhiên là **ước nguyên tố nhỏ nhất**, kí hiệu lp[x] (least prime). Trong Hình 1.1, màu của mỗi ô chính là lp của nó.

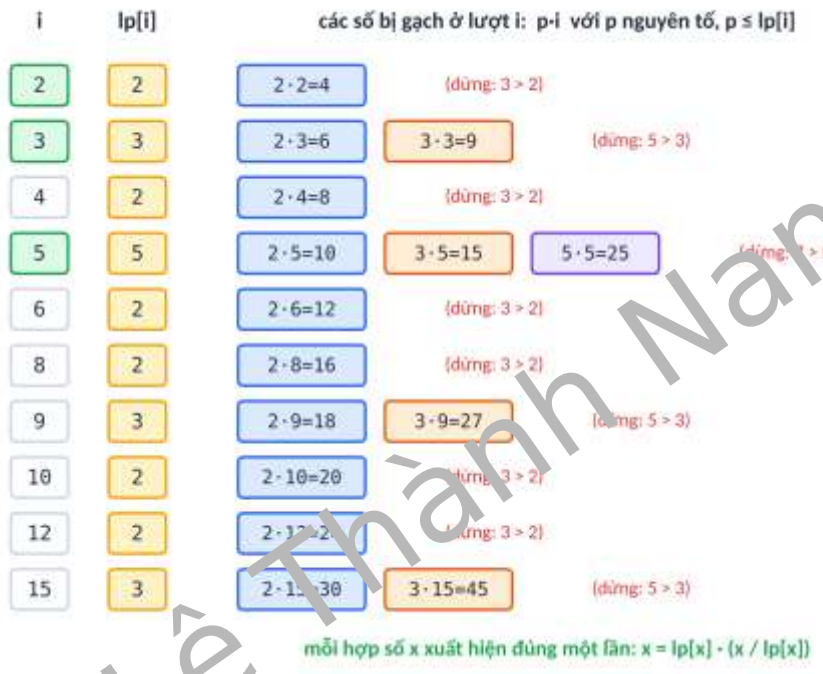
Lê Thành Nam

4. Sàng tuyến tính

Mọi hợp số x viết được **duy nhất** thành x = p·i với p = lp[x] và i = x/p. Khi đó i ≥ 2 và mọi ước nguyên tố của i đều không nhỏ hơn p, tức là **p ≤ lp[i]**. Ngược lại, nếu p nguyên tố và p ≤ lp[i] thì lp[p·i] = p. Vậy:

Mỗi hợp số x tương ứng đúng một cặp (i, p) với p nguyên tố, p ≤ lp[i], và x = p·i.

Sàng tuyến tính duyệt i = 2, 3, ..., n. Nếu lp[i] = 0 thì i chưa bị ai gạch, i nguyên tố: đặt lp[i] = i và thêm i vào danh sách số nguyên tố. Sau đó, với mỗi số nguyên tố p **từ nhỏ tới lớn** trong danh sách, chừng nào p ≤ lp[i] và p·i ≤ n, gạch p·i bằng cách đặt lp[p·i] = p.



Hình 1.3. *Sàng tuyến tính: ở lượt i chỉ gạch các số p·i với p ≤ lp[i]; mọi hợp số xuất hiện đúng một lần*

SANGNT.CPP: Sàng tuyến tính, O(n)

```
#include <bits/stdc++.h>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("SANGNT.INP", "r")) {
        freopen("SANGNT.INP", "r", stdin);
        freopen("SANGNT.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    // lp[x]: ước nguyên tố nhỏ nhất của x
    vector<int> lp(n + 1, 0);
    vector<int> pr;                          // các số nguyên tố đã tìm được
    for (int i = 2; i <= n; i++) {
        if (lp[i] == 0) { lp[i] = i; pr.push_back(i); }   // i nguyên tố
        for (int p : pr) {                   // gạch x = p·i với p ≤ lp[i]
            if (p > lp[i] || (long long)p * i > n) break;
            lp[p * i] = p;                   // mỗi hợp số bị gạch đúng một lần
        }
```

Lê Thành Nam

SANGNT.CPP: Sàng tuyến tính, O(n)

```
    }
    long long tong = 0;
    for (int p : pr) tong += p;
    cout << pr.size() << ' ' << tong << '\n';
    return 0;
}
```

■ SÀNG TUYẾN TÍNH ĐÚNG VÀ CHẠY TRONG O(n)

(a) Khi bắt đầu lượt i, lp[i] đã được tính đúng. (b) Mỗi hợp số x ≤ n được gán lp[x] đúng một lần. Tổng số bước của thuật toán là O(n).

■ CHỨNG MINH

(b) Theo nhận xét ở trên, hợp số x chỉ có thể bị gạch ở lượt i = x / lp[x], bởi p = lp[x]: ở mọi lượt khác, hoặc x không có dạng p·i, hoặc cặp (i, p) vi phạm điều kiện p ≤ lp[i]. Ngược lại ở lượt i = x/lp[x], vòng lặp xét các số nguyên tố p tăng dần từ 2 và chỉ dừng khi p > lp[i]; vì lp[x] ≤ lp[i] nên p = lp[x] chắc chắn được xét, x bị gạch. Mỗi bước của vòng lặp trong gạch một hợp số mới (trừ bước cuối cùng dừng vòng), nên tổng số bước không quá (số hợp số) + (số lượt) < 2n. (a) Nếu i là hợp số thì i = lp[i]·(i/lp[i]) với i/lp[i] < i, nên i đã bị gạch ở lượt i/lp[i] trước lượt i. Nếu i nguyên tố thì không ai gạch i (mọi số bị gạch đều là tích của hai số ≥ 2), nên lp[i] = 0 và ta nhận ra i nguyên tố.

■ CHẠY TAY TỚI 15

Danh sách số nguyên tố tăng dần trong quá trình chạy.

- i = 2: nguyên tố, p = {2}; gạch 2·2 = 4.
- i = 3: nguyên tố, p = {2, 3}; gạch 6, 9.
- i = 4: lp[4] = 2; gạch 8, rồi dừng vì 3 > 2 (12 sẽ bị gạch ở lượt i = 6 bởi 2).
- i = 5: nguyên tố; gạch 10, 15, (25 > 15).
- i = 6: lp = 2; gạch 12. i = 7: nguyên tố; gạch 14. Các lượt i ≥ 8 không gạch thêm số nào ≤ 15 (2·8 = 16 > 15).

Mỗi hợp số 4, 6, 8, 9, 10, 12, 14, 15 bị gạch đúng một lần.

Lê Thành Nam

5. Hai cách sàng trên thực tế

Sàng tuyến tính tối ưu về **số bước**: với n = 10^7, nó gạch 9.335.420 lần, ít hơn hẳn 22.850.051 lần của sàng Eratosthenes. Nhưng nó dùng mảng `int` lp (4 byte mỗi số) và truy cập bộ nhớ nhảy cóc, nên trên máy tính thật thường **không nhanh hơn**:

Chương trình	n = 10^6	n = 10^7	n = $3 \cdot 10^7$
Chia thử từng số	0,13 giây	3,28 giây	(quá lâu)

Chương trình	$n = 10^6$	$n = 10^7$	$n = 3 \cdot 10^7$
Eratosthenes, vector<bool>	0,0074 giây	0,065 giây	0,17 giây
Sàng tuyến tính	0,0087 giây	0,084 giây	0,30 giây

Giá trị thật sự của sàng tuyến tính nằm ở chỗ khác: **mảng lp**. Có lp, ta phân tích mọi số $\le n$ ra thừa số nguyên tố trong O(log n) (Bài 2), và tính được mọi **hàm nhân tính** (số ước, tổng ước, hàm phi Euler) cho cả đoạn 1..n trong O(n) (Bài 7). Khi chỉ cần biết số nào nguyên tố, sàng Eratosthenes với `vector<bool>` là lựa chọn đơn giản và hiệu quả.

■ GHI NHỚ

- Hợp số n có ước nguyên tố $\le \sqrt{n}$. Kiểm tra một số: $O(\sqrt{n})$; kiểm tra mọi số tới n bằng chia thử: quá chậm.
- Eratosthenes: với $p \le \sqrt{n}$ chưa bị gạch, gạch từ p^2 bước p. O(n log log n), dùng `vector<bool>`.
- Tuyến tính: x = lp[x]·i được gạch duy nhất ở lượt i; vòng trong dừng khi p > lp[i]. O(n), cho thêm mảng lp.

PHẦN BÀI TOÁN

■ BÀI TOÁN: SANGNT

Cho số nguyên n. Hãy đếm các số nguyên tố không vượt quá n và tính tổng của chúng.

Dữ liệu vào: Một dòng chứa n ($2 \le n \le 3 \cdot 10^7$).

Kết quả: Hai số: số lượng và tổng các số nguyên tố không vượt quá n.

Dữ liệu vào	Kết quả	Giải thích
10	4 17	Các số nguyên tố 2, 3, 5, 7.
100	25 1060	

6. Cách chậm: chia thử từng số

Chia thử từng số, O(n√n)

```cpp
#include <bits/stdc++.h>
using namespace std;

bool laNguyenTo(int x) {                    // chia thử tới căn bậc hai
    if (x < 2) return false;
    for (int d = 2; (long long)d * d <= x; d++)
        if (x % d == 0) return false;
    return true;
}

int main() {
    ios_base::sync_with_stdio(false);
```

Lê Thành Nam

Chia thử từng số, O(nVn)

```
    cin.tie(nullptr);
    int n;
    cin >> n;
    long long dem = 0, tong = 0;
    for (int x = 2; x <= n; x++)
        if (laNguyenTo(x)) { dem++; tong += x; }
    cout << dem << ' ' << tong << '\n';
    return 0;
}
```

7. Cách nhanh: sàng

Cả hai chương trình ở mục 2 và mục 4 đều giải được bài toán. Chương trình mẫu dùng sàng tuyến tính để chuẩn bị cho các bài học sau, nơi ta cần mảng lp.

■ ĐẾM SỐ PHÉP TÍNH

Với n = 10^7: chia thử mất 3,28 giây, sàng Eratosthenes 0,065 giây, sàng tuyến tính 0,084 giây. Với n = $3 \cdot 10^7$, sàng tuyến tính dùng mảng lp 120 MB và mất 0,30 giây; sàng Eratosthenes chỉ dùng 3,75 MB và mất 0,17 giây.

■ BẪY THƯỜNG GẶP

- **Tràn số khi tính p·i hay p·p:** với n gần $3 \cdot 10^7$, p·i có thể vượt 2^{31} trước khi so với n. Hãy ép sang `long long` khi so sánh.
- **Gạch từ 2p thay vì p^2:** vẫn đúng nhưng chậm hơn; ngược lại vòng ngoài `p <= n` thay cho `p * p <= n` không sai nhưng làm thừa nhiều bước.
- **Tổng các số nguyên tố tràn `int`:** tổng tới $3 \cdot 10^7$ cỡ $2{,}6 \cdot 10^{13}$.
- **Khai báo mảng $3 \cdot 10^7$ phần tử `int` trong hàm main:** tràn ngăn xếp. Dùng `vector` hoặc mảng toàn cục.

Lê Thành Nam

Luyện tập

Bài	Tên	Gợi ý liên hệ
1.1	NGUYÊN TỐ THỨ K	Sàng tới cận trên của số nguyên tố thứ k (khoảng k·(ln k + ln ln k)).
1.2	CẶP SONG SINH	Sàng rồi duyệt; dùng mảng cộng dồn để trả lời nhiều câu hỏi trên đoạn.
1.3	TỔNG HAI NGUYÊN TỐ	Đếm số cách viết số chẵn n ≤ $5 \cdot 10^7$ thành tổng hai số nguyên tố với bộ nhớ 64 MB: sàng số lẻ trên vector<bool>.

Tự kiểm tra nhanh:

1. Khi sàng Eratosthenes tới n = 100, những số nguyên tố p nào được dùng để gạch?

(Đáp số: Chỉ p = 2, 3, 5, 7 (vì $11^2 = 121 > 100$).)

2. Trong sàng tuyến tính, số $84 = 2^2 \cdot 3 \cdot 7$ bị gạch ở lượt i nào, bởi p nào?

(Đáp số: Ở lượt $i = 42$, bởi $p = 2 = lp[84]$.)

3. Vì sao vòng trong của sàng tuyến tính có thể dừng ngay khi $p > lp[i]$?

(Đáp số: Nếu $p > lp[i]$ thì $lp[p \cdot i] = lp[i] < p$, nên $p \cdot i$ phải được gạch bởi $lp[i]$ ở một lượt khác (lượt $p \cdot i / lp[i]$), không phải ở lượt này.)

Lê Thành Nam